

Documentación Técnica Arquitectónica: End-to-End AML Pipeline (Prevención de Lavado de Dinero)

Rol: Lead Data Engineer & Data Architect **Alcance del Documento:** Diseño de Arquitectura, Ciclo de Vida del Dato, Decisiones Estratégicas y Optimización.

1. Resumen Ejecutivo (Visión de Negocio y Tecnológica)

Descripción del Problema de Negocio

El lavado de dinero (AML - Anti-Money Laundering) representa uno de los desafíos más críticos y complejos para las instituciones financieras modernas. Los actores ilícitos utilizan tácticas sofisticadas de ofuscación, como el fraccionamiento de capitales (Structuring/Pitufeo), el uso de redes de cuentas puente (Fan-In/Fan-Out) y transacciones transfronterizas mediante criptoactivos (Mixers), para integrar fondos ilícitos en el sistema financiero legal. La detección tradicional basada en reglas estáticas sobre bases de datos relacionales ha demostrado ser ineficiente, generando una alta tasa de falsos positivos y siendo incapaz de procesar la volumetría transaccional actual en tiempos operativos viables.

Solución de Alto Nivel

Para mitigar estos riesgos de forma proactiva, hemos diseñado e implementado una arquitectura de datos moderna, escalable y en la nube. Esta solución ingesta, procesa y analiza volúmenes masivos de datos transaccionales crudos, transformándolos en inteligencia financiera procesable. Mediante la implementación del patrón de diseño "Medallion Architecture", garantizamos la gobernanza, calidad y trazabilidad de los datos en cada etapa, habilitando modelos de detección de anomalías y motores de reglas complejos que proveen alertas de alta precisión a los analistas de cumplimiento normativo (Compliance).

Stack Tecnológico Principal

- **Almacenamiento Transaccional y Origen (Raw):** Azure SQL Database.
 - **Procesamiento Distribuido y Transformación:** Databricks / Apache Spark (PySpark).
 - **Capa de Almacenamiento Analítico (Lakehouse):** Delta Lake.
 - **Generación de Reportes Automáticos:** Librería fpdf (Python) para la emisión programática de Reportes de Operaciones Sospechosas (ROS / SAR).
 - **Visualización e Inteligencia de Negocios:** Power BI / Databricks Dashboards para el rastreo del linaje de fondos.
-

2. Arquitectura de Datos y Ciclo de Vida del Dato (Profundidad Técnica)

La columna vertebral de este ecosistema de datos es la Arquitectura Medallón, que impone un rigor progresivo sobre la estructura y calidad de la información a medida que fluye por el pipeline.

Capa Bronce (Raw Data - Ingesta)

La Capa Bronce representa el punto de entrada de nuestro Lakehouse. Su objetivo principal es mantener un registro inmutable e histórico de los datos crudos exactamente como provienen del sistema transaccional origen.

- **Proceso de Ingesta:** Los datos transaccionales, que incluyen ruido sistémico intencional, valores nulos y formatos dispares, son extraídos desde nuestra base de datos relacional principal, específicamente desde la tabla `tabladefault.bronce_azure_local` alojada en Azure SQL.

- **Implementación Técnica:** Utilizamos conectores JDBC de PySpark en Databricks para realizar lecturas eficientes (bulk reads) o extracciones incrementales (CDC) de esta tabla origen. Los datos se depositan en el almacenamiento en la nube sin aplicar transformaciones estructurales (Schema-on-Read), conservando la fidelidad total del evento origen para propósitos de auditoría y reprocesamiento.

Capa Plata (Silver - Limpieza y Estandarización)

En esta fase, los datos transicionan de un estado "crudo" a un estado "listo para el análisis". La lógica de limpieza, orquestada mediante el script `capa_plata_AML`, es crucial para garantizar que los modelos posteriores no sufran de degradación (Garbage In, Garbage Out).

- **Transformaciones PySpark (Transform & Cleanse):**

- **Manejo de Nulos e Integridad Referencial:** Se ejecutan filtros estrictos para purgar registros que carecen de Identificadores Únicos de Transacción (Primary Keys nulas) utilizando `df.na.drop(subset=["PK_Transaccion"])`.
- **Normalización Financiera:** Este es un paso crítico. Los montos brutos, a menudo ingresados como *strings* o *floats* imprecisos, sufren un *casting* riguroso a tipo `DoubleType()` (o `DecimalType` dependiendo de la precisión del motor) y son redondeados matemáticamente a dos decimales exactos. Esto erradica errores de coma flotante que podrían desencadenar alertas de estructuración erróneas.
- **Formateo Temporal:** Las fechas de transacción (Timestamp) se estandarizan a un formato unificado (ej. `YYYY-MM-DD HH:mm:ss UTC`) para alinear los eventos cronológicamente, un requisito estricto para el análisis de series temporales en el pitufeo.
- **Estandarización Categórica:** Campos como tipos de moneda, códigos de sucursal y descripciones pasan por rutinas de limpieza (eliminación de espacios en blanco `trim()`, conversión a mayúsculas `upper()`).
- **Almacenamiento ACID con Delta Lake:** El DataFrame resultante no se guarda como un archivo Parquet estándar o CSV. Se persiste utilizando el formato Delta Lake en la tabla gestionada `aml_proyect.transacciones_plata_robert`. Esto otorga garantías transaccionales (ACID) sobre nuestro Data Lake. Permite realizar operaciones de Upsert (`MERGE INTO`) para manejar actualizaciones de estado de transacciones de forma segura y eficiente, evitando la duplicación de datos que arruinaría las métricas de lavado.

Capa Oro (Gold - Análisis de Negocio y AML)

La Capa Oro es el pináculo del pipeline; aquí el dato se convierte en producto. En el script `capa_oro_AML`, la tabla `transacciones_plata_robert` se cruza (`JOIN`) con las dimensiones maestras (ej. KYC de Clientes).

- **Agrupación y Agregación (Feature Engineering):** Mediante PySpark, agrupamos los datos a nivel de `ID_Cliente` (y ventanas temporales) usando `groupBy()`. Calculamos ventanas móviles, promedios móviles, conteo de transacciones por día y desviaciones estándar de los montos operados.
 - **Motor de Reglas y Banderas de Alerta (Flags):**
 - **Flag_Structuring_Pitufeo:** Se evalúa si el usuario ha realizado `$N$` transacciones en un periodo de `$T$` días, donde la suma total se aproxima sospechosamente al umbral legal de reporte (ej. `$9,900 USD` frente al límite de `$10,000 USD`), utilizando funciones de ventana de Spark (`Window.partitionBy()`).
 - **Flag_Fan_In:** Se detecta si múltiples cuentas de origen distintas transfieren montos a una única cuenta destino (embudo) en un corto periodo, seguido de una liquidación o retiro rápido.
 - **Flag_Crypto_Mixer:** Identifica patrones de transferencias hacia y desde identificadores conocidos de Exchanges centralizados o plataformas P2P, analizando saltos (hops) en el historial del cliente.
 - **Output Final:** El resultado de estos cálculos, sumado al score del modelo de Machine Learning, se persiste en la tabla altamente desnormalizada `aml_proyect.gold_perfiles_riesgo_cliente`, optimizada para lectura (BI) y consumo por parte del script de reporte `fpdf`.
-

3. Sección Especial de Preparación para Entrevistas Técnicas

Pregunta 1: Justificación del Modelo Medallion y el uso de Delta Lake

Respuesta del Arquitecto: Estructurar los datos en el paradigma Medallion (Bronze, Plata, Oro) no es un capricho estético; es una necesidad de gobernanza y mantenibilidad. Desacopla la ingesta, la limpieza y la lógica de negocio. Si una regla AML cambia mañana, solo reprocesamos la capa Oro leyendo desde la Plata, sin tener que volver a limpiar terabytes de datos crudos. Si descubrimos un error en nuestro algoritmo de limpieza, volvemos a leer de la Bronze (que retiene el histórico inmutable) y sobrescribimos la Plata.

Decidimos utilizar **Delta Lake** sobre bases de datos relacionales tradicionales o formatos Parquet simples por varias ventajas técnicas insuperables en un entorno Big Data: 1. **Transacciones ACID en Cloud Storage:** Al inyectar miles de transacciones diarias, necesitamos garantías de que una escritura a la mitad no corromperá la tabla analítica (Aislamiento y Atomicidad). Parquet puro no ofrece esto. 2. **Schema**

Enforcement y Evolution: En AML, si el sistema origen muta e inyecta una columna corrupta, Delta Lake rechaza la escritura (Schema Enforcement), protegiendo la integridad del modelo de detección. Al mismo tiempo, permite evolucionar el esquema (añadir nuevas métricas AML) sin bloquear operaciones. 3. **Time Travel (Auditoría Legal):** Fundamental en cumplimiento normativo. Si un ente regulador requiere ver el estado exacto de los datos y las alertas generadas hace 6 meses, Delta Lake permite hacer un `SELECT * FROM tabla TIMESTAMP AS OF 'fecha'`, garantizando trazabilidad absoluta.

Pregunta 2: El Recorrido, Transformación Detallada y Desafíos Técnicos

Respuesta del Arquitecto: El viaje del dato comienza cuando el Scheduler de Databricks activa el Job. PySpark abre una conexión JDBC hacia `tabladeault.bronce_azure_local` en Azure SQL, ejecutando una lectura paralelizada que ingesta el delta diario. Este *DataFrame* crudo se deposita temporalmente en la capa Bronze de nuestro Cloud Storage.

Inmediatamente inicia el pipeline Plata (capa_plata_AML). Aquí enfrentamos los **desafíos técnicos más significativos**: 1. **Manejo de PKs Nulos y Duplicidad:** Los sistemas legacy a menudo envían registros "sucios". La limpieza no puede ser un simple `dropna()`, requiere identificar si el nulo es un error sistémico o un dato corrupto. Usamos `dropDuplicates()` con una clave compuesta de hashes (`hash(monto, fecha, id_cliente)`) para erradicar reintentos del origen, asegurando que no reportemos falsos volúmenes al área legal. 2. **Sesgo de Datos (Data Skew):** En la transformación Plata hacia Oro, notamos un cuello de botella crítico. Ciertos clientes "ballenas" o cuentas institucionales tenían millones de transacciones, mientras que los perfiles normales tenían pocas. Al hacer el `groupBy("ID_Cliente")` en PySpark, esto generaba "Data Skew", sobrecargando pocos nodos trabajadores (stragglers) y extendiendo el tiempo de procesamiento horas. Tuvimos que intervenir la arquitectura. 3. **Precisión Financiera (Desafío de Punto Flotante):** Mantener la exactitud del centavo es crítico en umbrales de \$10,000. Forzamos el *casting* estricto de Spark SQL y aplicamos funciones `round()` deterministas, evitando que el *casting* implícito a Float generara valores como 9999.9999 que evadirían la regla exacta de 10000.00.

Pregunta 3: Aplicación Real del Valor del Negocio (Caso Numérico)

Respuesta del Arquitecto: La tabla `aml_proyect.gold_perfiles_riesgo_cliente` es el "Sistema Nervioso" del área de Compliance. Cambiamos el paradigma de analistas buscando agujas en pajares a analistas revisando expedientes pre-procesados.

Ejemplo Práctico de Pitufeo (Structuring): Supongamos el cliente *Juan Pérez*. Un analista manual vería transacciones aisladas. Nuestra Capa Gold, sin embargo, evaluó lo siguiente en una ventana móvil de 5 días: * Día 1: Depósito efectivo \$2,800 * Día 2: Depósito efectivo \$2,500 * Día 3: Depósito efectivo \$2,900 * Día 5: Depósito efectivo \$1,700 * **Total Consolidado en 5 días: \$9,900 USD.**

El script de la Capa Oro calculó el acumulado, cruzó la información con el umbral normativo (\$10,000 USD) y activó la alerta: `Flag_Structuring_Pitufeo = TRUE` y elevó el `AML_Risk_Score` a 85 (Alto Riesgo). Consecuencia: El analista recibe a primera hora en su bandeja de entrada el reporte PDF (generado por `fpdf`) detallando exactamente este flujo, con la instrucción automática de evaluar la emisión de un ROS a las autoridades, ahorrando días de investigación manual.

4. Optimización, Monitoreo y Escalabilidad

Para que este pipeline opere de manera confiable en un entorno de misión crítica, implementamos una robusta capa de observabilidad y estrategias avanzadas de *tuning*.

Monitoreo y Sistema de Alertas Automáticas

El ecosistema completo está orquestado mediante Databricks Workflows (Jobs). Hemos configurado notificaciones por correo electrónico vinculadas a los eventos del clúster: * **Notificaciones de Fallo (On Failure):** Si una tarea dentro de la DAG del pipeline (ej. falla la conexión JDBC a Azure SQL, o un error de memoria (OOM) en la Capa Plata) se aborta, el sistema dispara inmediatamente un correo con la traza de error (stack trace) y el Job ID al equipo de Data Engineering de guardia. * **Notificaciones de SLA (Duration Warning):** Si el pipeline suele tardar 45 minutos y excede los 90 minutos, se genera una alerta preventiva de degradación de rendimiento.

Optimización y Alertas de Rendimiento ("Insights" de Databricks)

Durante la fase de estabilización en producción, la pestaña "Insights" y la Spark UI revelaron áreas críticas de mejora: 1. **Derrame a Disco (Spill to Disk):** Al realizar cruces (JOINS) pesados entre la tabla de transacciones de un año y el maestro de clientes, identificamos *Disk Spill* (tanto de memoria como de disco). La memoria RAM de los ejecutores era insuficiente para mantener las particiones durante el "Shuffle". *

Estrategia de Refactorización: Optimizamos configurando `spark.sql.shuffle.partitions` dinámicamente en base al volumen de datos, y escalamos verticalmente los nodos (instancias optimizadas para memoria, ej. serie E en Azure). Además, aplicamos `broadcast(clientes_df)` ya que la tabla de dimensiones (clientes) era lo suficientemente pequeña para caber en memoria de cada nodo, erradicando el shuffle pesado de esa tabla. 2. **Sesgo de Datos (Data Skew):** Las operaciones de agregación y los JOINS sufrían de tareas que tardaban un 90% más que el promedio. * *Estrategia de Refactorización:* Implementamos **Adaptive Query Execution (AQE)** de Spark 3.x (`spark.sql.adaptive.enabled=true`, `spark.sql.adaptive.skewJoin.enabled=true`). Además, si el problema persistía en agrupaciones lógicas complejas, introduciríamos **"Salting"** (añadir una clave aleatoria temporal a la columna de cruce para forzar a Spark a distribuir la carga en los ejecutores).

Escalabilidad a Millones de Transacciones Diarias

La arquitectura actual está diseñada para escalar horizontalmente de manera transparente: * **Auto-scaling Clusters:** Los clústeres de Databricks están configurados con escalado automático (ej. de 2 a 16 nodos *workers*). El clúster absorbe automáticamente los picos de transaccionalidad (ej. fines de mes o black friday) sin intervención humana. * **Particionamiento Estratégico Delta:** La tabla `transacciones_plata_robert` debe particionarse físicamente en el almacenamiento subyacente. Para millones de registros, particionar por Año y Mes (`PARTITIONED BY (Year, Month)`) garantiza que cuando los analistas o modelos consulten datos recientes, Spark realice un "Partition Pruning", escaneando solo los fragmentos relevantes y reduciendo los tiempos de I/O en un 90%. * **Proyección a Streaming:** Si el negocio requiere detección en tiempo real, el pipeline está listo para evolucionar cambiando las lecturas batch por **Spark Structured Streaming**, leyendo directamente desde colas como Apache Kafka o Event Hubs, manteniendo casi intacta la lógica de la Capa Plata y Oro.

5. Conclusión

El proyecto **End-to-End AML Pipeline** trasciende la mera ingeniería de mover datos; es una herramienta de inteligencia y seguridad estratégica. Hemos logrado consolidar una arquitectura Medallion resiliente y tolerante a fallos, apalancada por las capacidades transaccionales de Delta Lake y el procesamiento masivo de PySpark.

El impacto es medible e inmediato: hemos automatizado la aplicación rigurosa de reglas de cumplimiento (estructuración, flujos cripto, inconsistencias KYC) sobre el 100% del universo transaccional, eliminando el error humano de los muestreos manuales. El resultado final son perfiles de riesgo limpios y reportes legales PDF emitidos sistemáticamente, permitiendo que el talento humano (analistas AML) enfoque su tiempo en la investigación de redes criminales de alto nivel, sustentados por una plataforma de datos altamente escalable, monitoreada y auditable.

6. Explicación del Código Fuente (Capa Bronce)

Esta sección desglosa el funcionamiento de los scripts principales responsables de la generación de datos y la inyección de ruido.

config.py (Configuración Centralizada)

Este módulo actúa como el archivo de configuración global. Define constantes críticas como el límite legal para la detección de pitufeo (`UMBRAL_ALERTA = 10000.0`), la probabilidad de inyectar registros defectuosos (`PROBABILIDAD_RUIDO = 0.15`), las tasas de cambio de divisas y la cadena de conexión cifrada (ODBC) para comunicarse con Azure SQL Database. Centralizar esto permite cambiar parámetros del sistema (ej. el volumen de generación, o contraseñas vía variables de entorno) sin tocar la lógica dura, manteniendo el código limpio y parametrizable.

data_quality.py (Inyección de Ruido)

Aquí simulamos los defectos naturales de un entorno bancario real para estresar intencionalmente la Capa Plata. La función `inyectar_ruido(df)` recibe un DataFrame limpio y utiliza operaciones vectorizadas de numpy (como `np.random.rand`) para marcar el 15% de los registros para inyección de ruido. Estos registros se dividen en 4 tipos de errores, probando diferentes estrategias de limpieza: 1. **Alteración de fecha:** Cambia el formato de YYYY-MM-DD a DD/MM/YYYY. 2. **Errores tipográficos:** Sustituye la moneda correcta ("USD") por valores sucios ("usd", " UDS ", "US \$"). 3. **Nulos Forzados:** Anexa filas, estableciendo su `PK_Transaccion` como nula (`np.nan`) para probar la integridad referencial. 4. **Duplicidad:** Clona filas desplazando la fecha un segundo hacia el futuro. Este módulo garantiza que nuestro pipeline sea resiliente y que los algoritmos de Spark en la capa Plata verdaderamente limpien estos errores antes de que los modelos de ML puedan verse afectados.

main.py (Orquestador de Capa Bronce)

Es el script principal de punto de entrada que coordina el proceso de simulación transaccional: 1.

Generación Aleatoria: Selecciona perfiles financieros (ej. 70% Clientes Normales, y un 10% para cada tipología criminal: Pitufos, Lavador Plataformas, Lavador Cripto) de forma probabilística (usando `random.choices`) hasta alcanzar una meta de registros dinámica (entre 9,500 y 10,500). 2. **Ejecución de Objetos:** Llama al método `generar_transacciones` de cada clase instanciada (apoyándose en POO), recolectando operaciones históricas desde hace 60 días hacia atrás. 3. **Llamada al Ruido:** Invoca a `inyectar_ruido` pasando el DataFrame consolidado para "ensuciar" la información. 4. **Exportación y Subida:** Finalmente, guarda un respaldo en formato CSV a nivel local (para control) y utiliza el motor de SQLAlchemy (definido en `db_connection.py`) para insertar ("append") la información masivamente en las tablas `Fact_Transacciones` y `Dim_Cliente` de Azure SQL Database, quedando lista para que Databricks comience su orquestación.